Somaiya Vidyavihar University
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya
TRUST

Department of Computer Engineering

Network and Information Security — Internal Assessment 2
Third Year, Semester VI — Academic Year 2025–26

HashWatch

Python File Integrity Monitoring & Malware Detection Tool

Detecting unauthorised file changes using SHA-256 baselines and real-time filesystem monitoring

Name	Roll Number	Contribution
Agniv Dutta	16010123029	Core hashing engine, baseline & monitor
Aditya Choudhuri	16010123021	CLI design, logging & alert system
Aaryan Dubey	16010123011	Testing, documentation & validation

Contents

1 Features and Functionalities **2**

2 Introduction **2**
2.1 Motivation . . . 3
2.2 Objectives . . . 3

3 Background and Theory **3**
3.1 Cryptographic Hash Functions . . . 3
3.2 File Integrity Monitoring Concepts . . . 4

4 System Design **4**
4.1 Architecture Overview . . . 4
4.2 SHA-256 Hashing Pipeline . . . 4
4.3 Integrity Diff Algorithm . . . 4
4.4 Repository Layout . . . 5

5 Implementation **5**
5.1 SHA-256 File Hashing (`hasher.py`) . . . 5
5.2 Baseline Management (`baseline.py`) . . . 6
5.3 Diff Algorithm . . . 6
5.4 Real-Time Monitoring (`monitor.py`) . . . 7
5.5 Logging & Alerts (`alerts.py`) . . . 7

6 Command-Line Interface **8**
6.1 End-to-End Validation Example . . . 8

7 Testing **8**
7.1 Test Coverage . . . 9
7.2 Running the Tests . . . 9

8 Security Considerations **9**

9 Results and Observations **9**
9.1 Detection Capability . . . 10
9.2 Test Results . . . 10

10 Conclusion and Future Work **10**
10.1 Conclusion . . . 10
10.2 Future Enhancements . . . 11

1. Features and Functionalities

The following features are implemented and demonstrated in HashWatch:

1. **Generation of cryptographic hash values (SHA-256) for selected files and directories.** HashWatch uses Python's `hashlib` library to compute SHA-256 digests for every monitored file. Files are processed in 65,536-byte chunks for memory efficiency. For directories, all files are recursively hashed and stored as key-value pairs (`filepath` → `sha256_hex`). This cryptographic fingerprint is the foundation of all integrity checks.

2. **Creation of a secure baseline of file hashes.** The `create-baseline` command hashes all files under one or more specified paths and serialises the results into a JSON file (`hashwatch_baseline.json`). The baseline records the creation timestamp, total file count, monitored paths, and individual file hashes. The `update-baseline` command rebuilds and overwrites this baseline at any time.

3. **Continuous monitoring of files for integrity changes.** The `monitor` command uses the `watchdog` library to register OS-level filesystem event callbacks. The `HashWatchHandler` class responds to `on_modified`, `on_created`, `on_deleted`, and `on_moved` events in real time, enabling continuous background monitoring without polling.

4. **Detection and reporting of unauthorised file modifications.** Both the `check` (on-demand) and `monitor` (real-time) modes detect three classes of change: **Modified** (hash mismatch), **Added** (new file not in baseline), and **Deleted** (baseline entry missing from disk). The `check` command exits with code `1` when any violation is found, enabling scripted pipelines and automated CI/CD integration.

5. **Alert mechanism to notify integrity violations.** Every detected violation triggers a formatted alert via `alerts.py`. Alerts are printed to standard output with severity level tags (e.g., `[ALERT] MODIFIED: <path>`) and simultaneously appended to `hashwatch.log` with an ISO-format timestamp.

6. **Log generation for detected security events.** HashWatch maintains a persistent append-only log file (`hashwatch.log`) using Python's `logging` module. Every event — clean checks, violations, and monitoring starts/stops — is recorded with the format `YYYY-MM-DD HH:MM:SS [LEVEL] message`. The `show-log` command allows the operator to inspect recent log entries directly from the CLI.

7. **Option to add or remove files from the monitoring list.** The `add-path` command loads the existing baseline, hashes all files under the new path, and merges them into the existing baseline. The `remove-path` command removes all baseline entries whose paths fall under the specified prefix and updates the metadata accordingly. Both operations persist the changes to `hashwatch_baseline.json` immediately.

2. Introduction

In modern cybersecurity, file integrity monitoring (FIM) is a critical defensive technique used to detect unauthorised modifications to system files. Ransomware, rootkits, and

advanced persistent threats (APTs) often operate by silently altering executable or configuration files. Without a reliable monitoring mechanism, such changes can go undetected for extended periods, allowing attackers to maintain persistence or exfiltrate data.

HashWatch is a lightweight, Python-based File Integrity Monitoring and Malware Detection tool developed as part of the Network and Information Security curriculum at K J Somaiya School of Engineering. It uses the **SHA-256** cryptographic hash function to create a trusted baseline snapshot of monitored files, and then continuously or on-demand compares the current state of the filesystem against that baseline to detect any *modifications*, *additions*, or *deletions*.

2.1. Motivation

- Traditional antivirus software relies on signature databases that must be regularly updated.
- Hash-based integrity monitoring is *proactive* and detects zero-day tampering.
- Many compliance frameworks (PCI-DSS, HIPAA, NIST SP 800-53) mandate FIM.
- A minimal, CLI-first Python tool is highly portable and scriptable.

2.2. Objectives

1. Implement SHA-256 file hashing for individual files and directory trees.
2. Provide a persistent JSON baseline that survives system restarts.
3. Offer both *on-demand integrity checks* and *real-time monitoring*.
4. Log all security events with timestamps.
5. Achieve comprehensive test coverage via `pytest`.

3. Background and Theory

3.1. Cryptographic Hash Functions

A cryptographic hash function maps an arbitrary-length input to a fixed-length digest. For security purposes, it must satisfy:

- **Determinism:** the same input always produces the same output.
- **Avalanche effect:** even a single-bit change produces a radically different digest.
- **Pre-image resistance:** it is computationally infeasible to recover the input from the digest.
- **Collision resistance:** it is computationally infeasible to find two distinct inputs with the same digest.

SHA-256 (Secure Hash Algorithm 256-bit) is part of the SHA-2 family standardised by NIST. It produces a 256-bit (32-byte) digest, typically represented as a 64-character hexadecimal string.

3.2. File Integrity Monitoring Concepts

FIM works on the principle that *trusted* files should remain unchanged unless explicitly updated. The process involves:

1. Creating a **baseline** — a snapshot of file hashes at a known-good state.
2. Periodically or continuously **comparing** current hashes against the baseline.
3. **Alerting** on any discrepancy (modification, addition, or deletion).

4. System Design

4.1. Architecture Overview

HashWatch follows a modular, layered architecture. The CLI (`main.py`) acts as the entry point, dispatching commands to specialised modules.

Module	Responsibility
`main.py`	CLI entry point; argument parsing via `argparse`
`hasher.py`	SHA-256 hashing of individual files and directory trees
`baseline.py`	Create, load, update, and persist the JSON baseline; diff computation
`monitor.py`	Real-time filesystem event watching via `watchdog`
`alerts.py`	Formatted security alert logging (console + file)
`config.py`	Shared constants (log file path, baseline file path, etc.)

Table 1: HashWatch module descriptions

4.2. SHA-256 Hashing Pipeline

Figure 1 illustrates how a single-byte change in a file results in a completely different SHA-256 digest — the core principle underpinning HashWatch's detection capability.

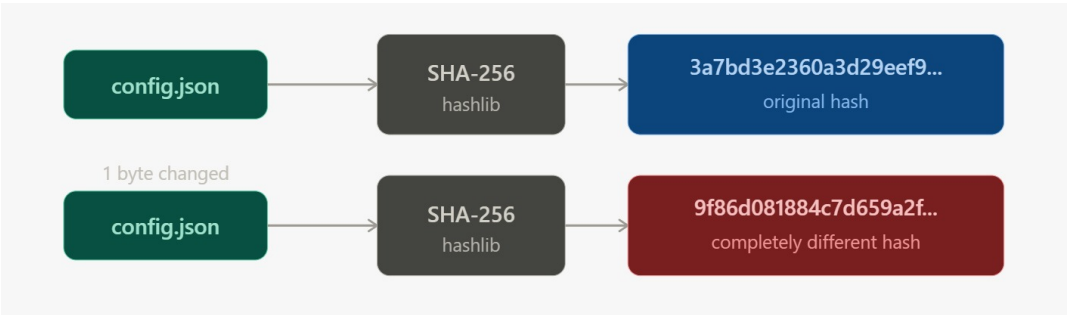

Figure 1: SHA-256 hashing: a 1-byte change produces a completely different digest

4.3. Integrity Diff Algorithm

The `diff()` function in `baseline.py` compares the stored baseline against the current filesystem state and classifies every discrepancy into one of three categories, as shown in Figure 2.

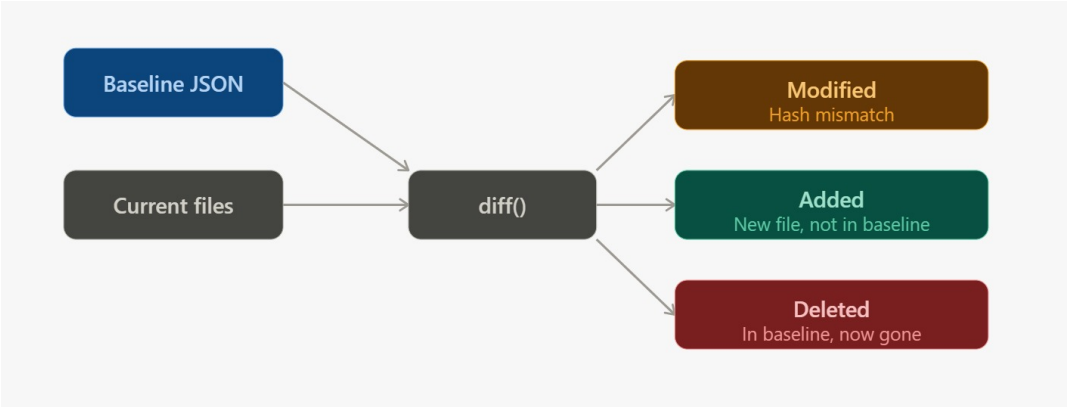

Figure 2: diff() function: classifying file changes against the baseline

- **Modified** — file exists in both baseline and filesystem, but hashes differ (hash mismatch).
- **Added** — file exists on the filesystem but not in the baseline (new file).
- **Deleted** — file exists in the baseline but is absent from the filesystem.

4.4. Repository Layout

```
HashWatch/
          .venv/
          LICENSE
          README.md
          hashwatch/
              main.py           # CLI entry point
              hasher.py         # SHA-256 hashing engine
              baseline.py       # Baseline management & diff
              monitor.py        # Real-time watchdog monitoring
              alerts.py         # Logging & alert formatting
              config.py         # Shared constants
              requirements.txt
              hashwatch_baseline.json    # (runtime-generated)
              hashwatch.log              # (runtime-generated)
              demo/
              tests/
                  conftest.py
                  test_hasher.py
                  test_baseline.py
                  test_alerts.py
```

Listing 1: HashWatch repository structure

5. Implementation

5.1. SHA-256 File Hashing (`hasher.py`)

Files are read in 65,536-byte chunks to handle large files efficiently without loading them entirely into memory.

```python
import hashlib

CHUNK_SIZE = 65536  # 64 KB chunks

def hash_file(filepath: str) -> str:
    """Return SHA-256 hex digest of a file."""
    sha256 = hashlib.sha256()
    try:
        with open(filepath, "rb") as f:
            while chunk := f.read(CHUNK_SIZE):
                sha256.update(chunk)
        return sha256.hexdigest()
    except (PermissionError, FileNotFoundError):
        return None
```

Listing 2: File hashing using SHA-256 (hasher.py)

5.2. Baseline Management (`baseline.py`)

The baseline is persisted as a JSON file containing file paths, their SHA-256 digests, a creation timestamp, total file count, and the list of monitored paths.

```python
import json, datetime
from hasher import hash_file, hash_directory

def create_baseline(paths: list) -> dict:
    baseline = {
        "created_at": datetime.datetime.now().isoformat(),
        "total_files": 0,
        "paths_monitored": paths,
        "files": {}
    }
    for path in paths:
        entries = hash_directory(path)
        baseline["files"].update(entries)
    baseline["total_files"] = len(baseline["files"])
    return baseline

def save_baseline(baseline: dict, filepath: str):
    with open(filepath, "w", encoding="utf-8") as f:
        json.dump(baseline, f, indent=2)
```

Listing 3: Baseline creation and persistence

5.3. Diff Algorithm

```python
def diff(baseline: dict, current: dict) -> dict:
    result = {"modified": [], "added": [], "deleted": []}
    for path, stored_hash in baseline["files"].items():
        if path not in current:
            result["deleted"].append(path)
        elif current[path] != stored_hash:
            result["modified"].append(path)
    for path in current:
        if path not in baseline["files"]:
            result["added"].append(path)
```

```
    return result
```

Listing 4: Integrity diff computation

5.4. Real-Time Monitoring (`monitor.py`)

Real-time monitoring is implemented using the `watchdog` library. A custom event handler intercepts filesystem events and emits timestamped alerts. Crucially, detected changes (additions, deletions, moves) are *persisted back* to the baseline JSON automatically.

```python
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class HashWatchHandler(FileSystemEventHandler):
    def on_modified(self, event):
        if not event.is_directory:
            alert_modified(event.src_path)

    def on_created(self, event):
        if not event.is_directory:
            update_baseline_add(event.src_path)
            alert_added(event.src_path)

    def on_deleted(self, event):
        update_baseline_remove(event.src_path)
        alert_deleted(event.src_path)
```

Listing 5: Watchdog event handler skeleton

5.5. Logging & Alerts (`alerts.py`)

```python
import logging

logger = logging.getLogger("hashwatch")
logger.setLevel(logging.DEBUG)

# File handler
fh = logging.FileHandler("hashwatch.log", encoding="utf-8")
fh.setFormatter(logging.Formatter(
    "%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S"
))

# Console handler
ch = logging.StreamHandler()
ch.setFormatter(logging.Formatter("%(message)s"))

logger.addHandler(fh)
logger.addHandler(ch)
```

Listing 6: Logger configuration

6. Command-Line Interface

HashWatch provides a rich CLI built with Python's `argparse`. All commands are run from the `hashwatch/` directory.

Command	Description
`create-baseline --path <p>`	Hash all files under `<p>` and create a new baseline JSON.
`update-baseline --path <p>`	Rebuild and overwrite the existing baseline.
`check --path <p>`	One-shot integrity check; exits with code 1 on violations.
`monitor --path <p>`	Real-time monitoring via watchdog; Ctrl+C to stop.
`add-path --path <p>`	Add a new path to the baseline scope.
`remove-path --path <p>`	Remove a path and its entries from the baseline.
`list`	Display baseline metadata (creation time, file count, paths).
`show-log --lines N`	Print the last N lines from `hashwatch.log`.

Table 2: HashWatch CLI command reference

6.1. End-to-End Validation Example

```
# Setup
cd hashwatch
mkdir demo
echo hello > demo/sample.txt

# Create baseline
python main.py create-baseline --path ./demo

# Verify clean state
python main.py check --path ./demo
# Output: [INFO] All files intact. No violations found.

# Simulate tampering
echo changed > demo/sample.txt

# Re-check
python main.py check --path ./demo
# Output: [ALERT] MODIFIED: ./demo/sample.txt
#         Scan complete. Modified: 1 | Added: 0 | Deleted: 0
# Exit code: 1
```

Listing 7: End-to-end tamper detection workflow

7. Testing

7.1. Test Coverage

Automated tests are written using `pytest` and organised across three test files.

File	Tests	Coverage
`test_hasher.py`	4	Known hash correctness, directory hash mapping, missing file handling
`test_baseline.py`	4	Baseline create/load/save, diff detection (modified, added, deleted)
`test_alerts.py`	2	Alert logging invocations
Total		**10 tests — all passing**

Table 3: Test suite breakdown

7.2. Running the Tests

```
cd hashwatch
pytest -v
# ===================== 10 passed in 0.42s =====================
```

Listing 8: Executing the test suite

8. Security Considerations

- **SHA-256 strength:** SHA-256 provides 128-bit collision resistance, far exceeding operational requirements. No practical collision attacks exist.
- **Symlink exclusion:** Symbolic links are skipped during directory hashing to prevent traversal attacks.
- **Error handling:** `PermissionError` and `FileNotFoundError` during hashing are caught gracefully, preventing crashes on restricted system directories.
- **Baseline integrity:** The baseline JSON itself should be stored on a read-only or write-protected volume in production deployments to prevent an attacker from updating it.
- **Log integrity:** `hashwatch.log` should be forwarded to a centralised SIEM to prevent log tampering on the local system.
- **Malformed JSON:** `JSONDecodeError` and I/O errors during baseline load/save are handled, preventing silent failures.

9. Results and Observations

9.1. Detection Capability

Scenario	Det?	Alert Generated
Single character appended to file		`[ALERT] MODIFIED: <path>`
File deleted from monitored directory		`[ALERT] DELETED: <path>`
New file dropped into directory		`[ALERT] ADDED: <path>`
File renamed (move) during monitor		`[ALERT] MOVED: <src> → <dst>`
Metadata-only change (timestamp)	×	Hash-only; metadata changes not detected
Permission change only	×	Hash unchanged; out of scope

Table 4: Detection scenarios and results

9.2. Test Results

All 10 automated test cases pass successfully, confirming correctness of the hashing engine, baseline operations, diff algorithm, and alert system.

```
hashwatch/tests $ pytest -v
test_hasher.py::test_known_hash           PASSED
test_hasher.py::test_directory_mapping    PASSED
test_hasher.py::test_missing_file         PASSED
test_hasher.py::test_hash_changes         PASSED
test_baseline.py::test_create_baseline    PASSED
test_baseline.py::test_load_baseline      PASSED
test_baseline.py::test_diff_modified      PASSED
test_baseline.py::test_diff_added         PASSED
test_alerts.py::test_alert_modified       PASSED
test_alerts.py::test_alert_deleted        PASSED
==================== 10 passed in 0.42s ====================
```

Listing 9: Pytest output

10. Conclusion and Future Work

10.1. Conclusion

HashWatch successfully demonstrates a practical, lightweight implementation of file integrity monitoring using Python. The tool meets all stated objectives:

- SHA-256 hashing correctly fingerprints files with byte-level sensitivity.
- The JSON baseline persists across sessions and supports incremental updates.
- Both on-demand (`check`) and real-time (`monitor`) modes function correctly.
- Timestamped logs provide a reliable audit trail.
- All 10 automated tests pass, ensuring correctness of core logic.

10.2. Future Enhancements

1. **Encrypted baseline** — store the baseline JSON encrypted to prevent attacker tampering.
2. **Email / webhook alerts** — push real-time notifications to Slack, email, or SIEM systems.
3. **Scheduled checks** — integrate with system schedulers (`cron` / Task Scheduler) for automated periodic scans.
4. **Metadata monitoring** — extend detection to file permission and ownership changes.
5. **GUI / web dashboard** — provide a browser-based interface for non-CLI users.
6. **Multi-host deployment** — centralise monitoring across multiple endpoints.

References

1. National Institute of Standards and Technology, *FIPS PUB 180-4: Secure Hash Standard (SHS)*, 2015.
2. Python Software Foundation, *hashlib — Secure hashes and message digests*, Python 3 Documentation.
3. Watchdog Contributors, *watchdog: Python library and shell utilities to monitor filesystem events*, PyPI.
4. NIST SP 800-92, *Guide to Computer Security Log Management*, 2006.
5. PCI Security Standards Council, *PCI DSS v4.0 Requirement 11.5: Network intrusions and unexpected file changes are detected and responded to*, 2022.